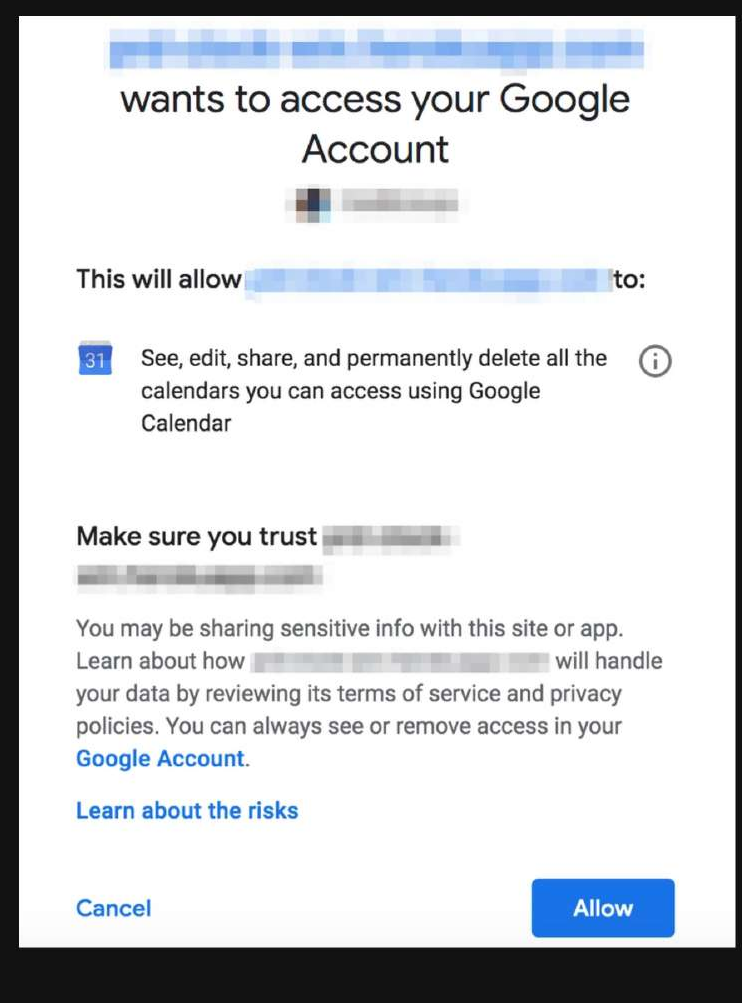


[redacted] wants to access your Google Account

[redacted]

This will allow [redacted] to:

See, edit, share, and permanently delete all the calendars you can access using Google Calendar

Make sure you trust [redacted]

You may be sharing sensitive info with this site or app. Learn about how [redacted] will handle your data by reviewing its terms of service and privacy policies. You can always see or remove access in your Google Account.

Learn about the risks

Cancel
Allow

OAuth 2.0

Imagine you have an AI photo editing app called AI Photo Editor.

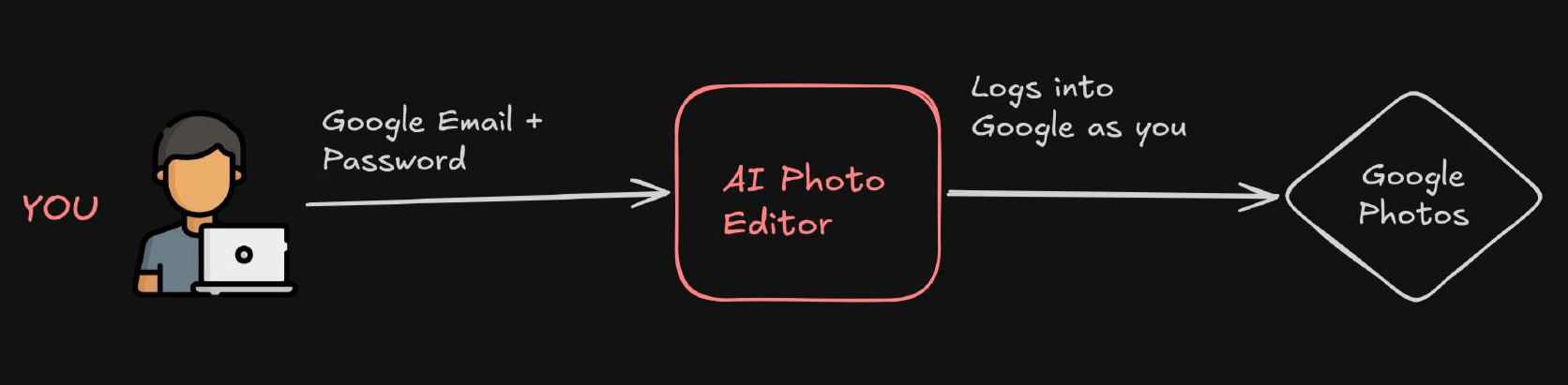
It lets users edit photos stored in Google Photos.

Question:

How can this app access your Google Photos without ever asking for your Google password?

That's the problem OAuth 2.0 solves.

The Problem Before OAuth


YOU
Google Email + Password
AI Photo Editor
Logs into Google as you
Google Photos

Problems:

App knows your password.
Password can be leaked.
App can access everything.
Hard to revoke only one app.
If you change password, every connected app breaks.

So we need a better solution.

The Idea Behind OAuth

Instead of sharing your password,

Google says:

"Don't give the app your password."

Instead:

"I'll give it a temporary permission."

That temporary permission is called an Access Token.

What is OAuth 2.0?

OAuth 2.0 is an authorization framework that lets one application obtain limited access to another application's resources on behalf of a user, without exposing the user's password.

Example:

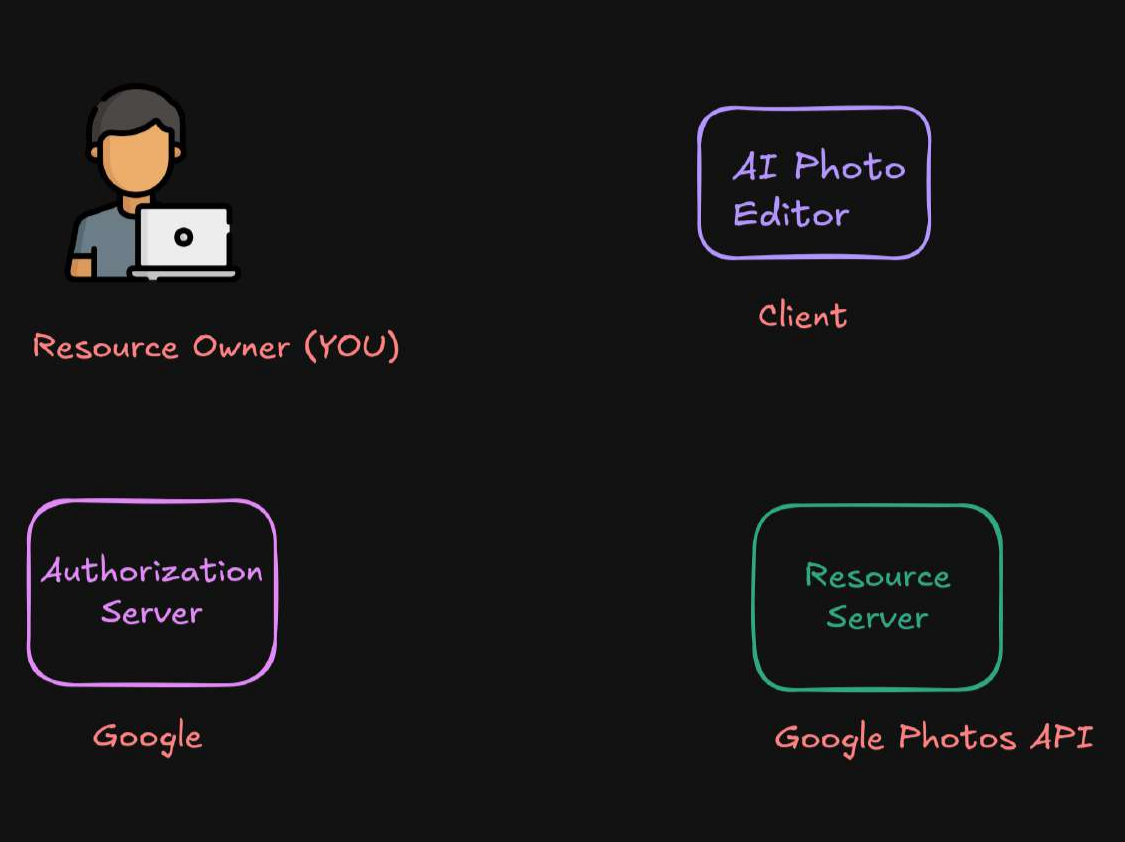
You use:

- Google Drive
- Dropbox
- GitHub
- Spotify
- Microsoft

An application can request only the permissions it needs, such as:

- Read your profile
- Read your email
- Read Photos
- Read calendar

OAuth Actors


Resource Owner (YOU)
AI Photo Editor
Client
Authorization Server
Google
Resource Server
Google Photos API

What is a Protected Resource?

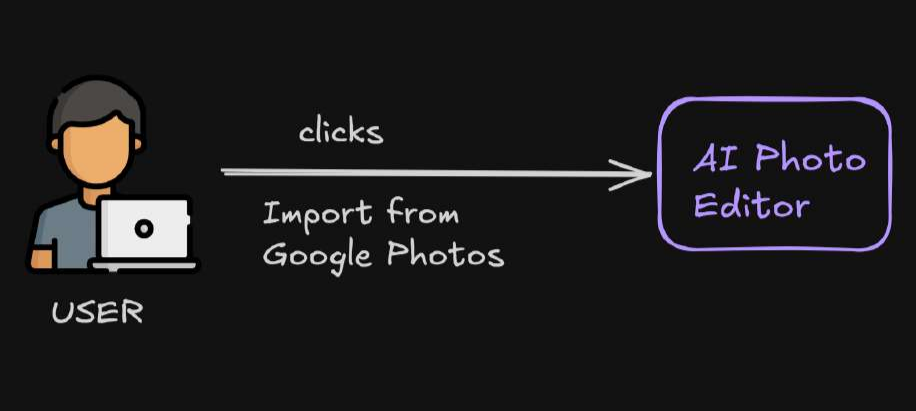
Your photos.
Videos.
Albums.
Metadata.

All these resources require permission.

Hence:
Protected Resources

Complete Authorization Code Flow

Step 1


USER
clicks
Import from Google Photos
AI Photo Editor

Step 2

Photo Editor app redirects browser.

accounts.google.com ....

Request contains

Client ID
Redirect URI
Scope
Response Type = code
State

Client ID

A public identifier that uniquely identifies the application requesting access.

Every OAuth application registered with Google gets a unique Client ID.

Google immediately knows:

Which application is making the request?
Who owns this application?
Which redirect URIs belong to it?
Which OAuth settings are configured

Why is it needed?

Imagine Google receives millions of OAuth requests every minute.

Without Client ID, Google wouldn't know:

Is this Canva?
Is this LinkedIn?
Is this Spotify?
Is this AI Photo Editor app?

The Client ID identifies the app.

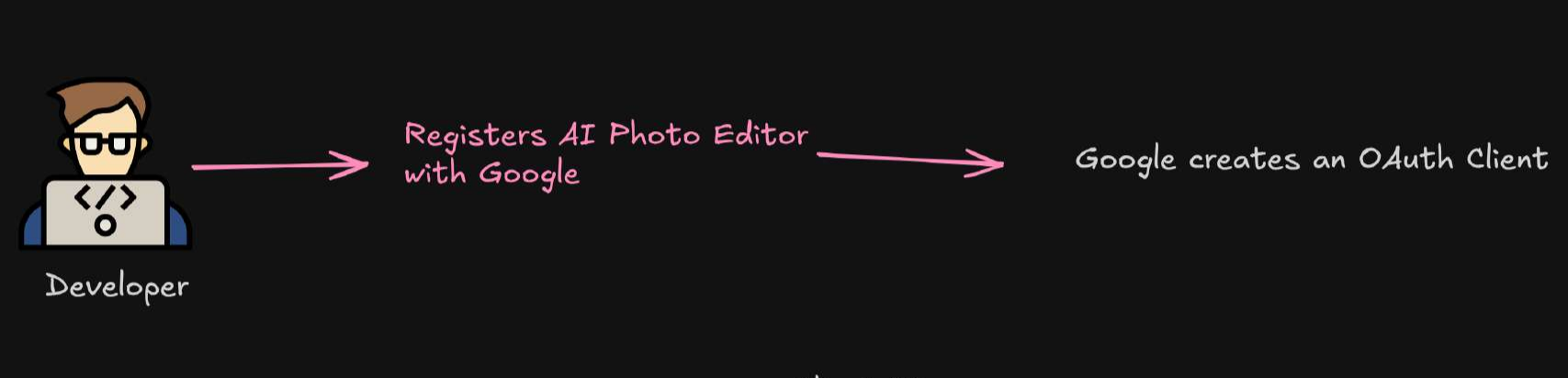
Is Client ID secret?

No.

Anyone can see it.
It travels in the browser URL.
It's similar to a username.
The secret part is the Client Secret, which is sent later by the backend.

Before OAuth Can Even Start

There is a one-time setup done by the application's developer.


Developer
Registers AI Photo Editor with Google
Google creates an OAuth Client

After registration, Google gives the application:

Client ID

Client Secret

Why is registration required?

Imagine Google didn't require registration.
Anyone could send this request:

GET /authorize

client_id=anything

Google would have no idea:

Who owns this app?
Is it trustworthy?
Where should users be redirected?
Which permissions is it allowed to request?

Registration solves all of this.

One correction to a common misconception

Many people say:

"Google generates a Client ID every time OAuth runs."

That's incorrect.

The Client ID is generated only once, when the developer registers the application.

Every user of that application uses the same Client ID.

Redirect URI

A Redirect URI is the URL where the Authorization Server (Google) sends the user's browser after the user approves or denies the permission request.

Google doesn't trust the Redirect URI in the request alone.

When AI Photo EDitor registered with Google, it already told Google:

Allowed Redirect URIs

https://aiphotoeditor.com/oauth/callback
https://app.photomagic.com/oauth/callback

If the request contains:

redirect_uri=https://evil.com/callback

Google immediately rejects it.

Scope

scope=https://www.googleapis.com/auth/photoslibrary.readonly

Scope tells Google:

"Exactly what permission is this application requesting?"

Not

Everything.

Only specific permissions.

It doesn't need

Gmail
Calendar
Contacts

It only needs

Google Photos.

Response Type

This tells Google:

"What should I return after the user grants permission?"

For Authorization Code Flow

response_type=code

means

Return an

Authorization Code

NOT an Access Token.

Why not send Access Token directly?

Security.
Access Token is powerful.

Instead

Browser only receives
Authorization Code

Backend exchanges it securely.

Why is this secure?

Because only the backend knows the:

Client Secret

The explanation above is 100% correct for traditional server-side web applications (confidential clients).

However, modern Single Page Applications (React, Angular, Vue) and mobile apps often do receive the Access Token directly, but only when using Authorization Code Flow with PKCE.

State

a random value generated by the client before redirecting the user to Google.

Example

state = A7xP92Lm

Photo app stores it.

Then redirects user.

Later

Google redirects back

https://photomagic.com/callback

?code=ABC123

&state=A7xP92Lm

Match?

Proceed.

Different?

Reject.

Why?

It prevents CSRF (Cross-Site Request Forgery) attacks.

Without the state parameter, an attacker could try to trick your application into accepting an OAuth response that belongs to someone else.
By generating a random value before the redirect and verifying that the same value comes back, the application knows the response corresponds to the OAuth request it originally initiated.

Why are all these values sent in the browser URL? Isn't that insecure?

Client ID → Public identifier, so exposing it is fine.
Redirect URI → Public and validated against the registered list.
Scope → Just requested permissions.
Response Type → Only tells Google what kind of response is expected.
State → A random, temporary value with no sensitive information.

The sensitive values: such as the Client Secret, Access Token, and Refresh Token -> are not sent in this initial browser redirect.
They are exchanged later through a secure server-to-server request between your backend and Google's token endpoint.

Step 3

Google authenticates user.

Password
OR
Passkey
OR
Fingerprint
OR
Face ID

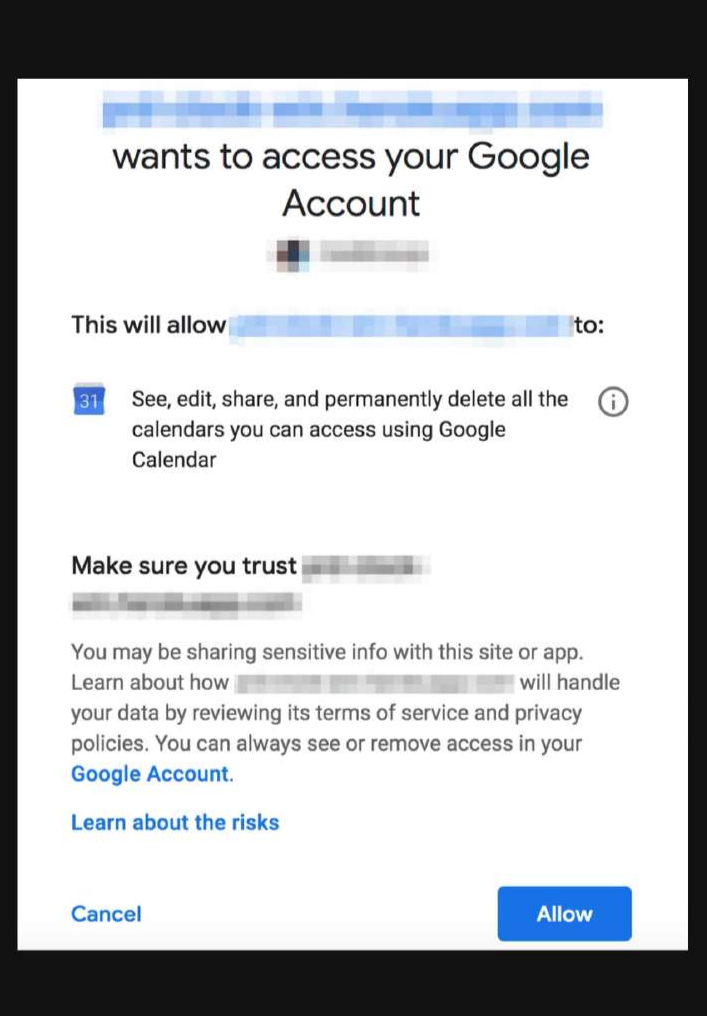
OAuth doesn't care how.

Google simply ensures:

"This is really Aniket."

Step 4

Consent Screen



[redacted] wants to access your Google Account

[redacted]

This will allow [redacted] to:

See, edit, share, and permanently delete all the calendars you can access using Google Calendar

Make sure you trust [redacted]

You may be sharing sensitive info with this site or app. Learn about how [redacted] will handle your data by reviewing its terms of service and privacy policies. You can always see or remove access in your Google Account.

Learn about the risks

Cancel
Allow

AI Photo Editor wants permission to

-> View Google Photos

Step 5

Authorization Code

Google generates

ABCD123XYZ

temporary
short-lived
one-time use

Very important.

Token Exchange

Backend contacts Google.

POST /token

Sends

Authorization Code
Client ID
Client Secret
Redirect URI

Google verifies everything.

Returns

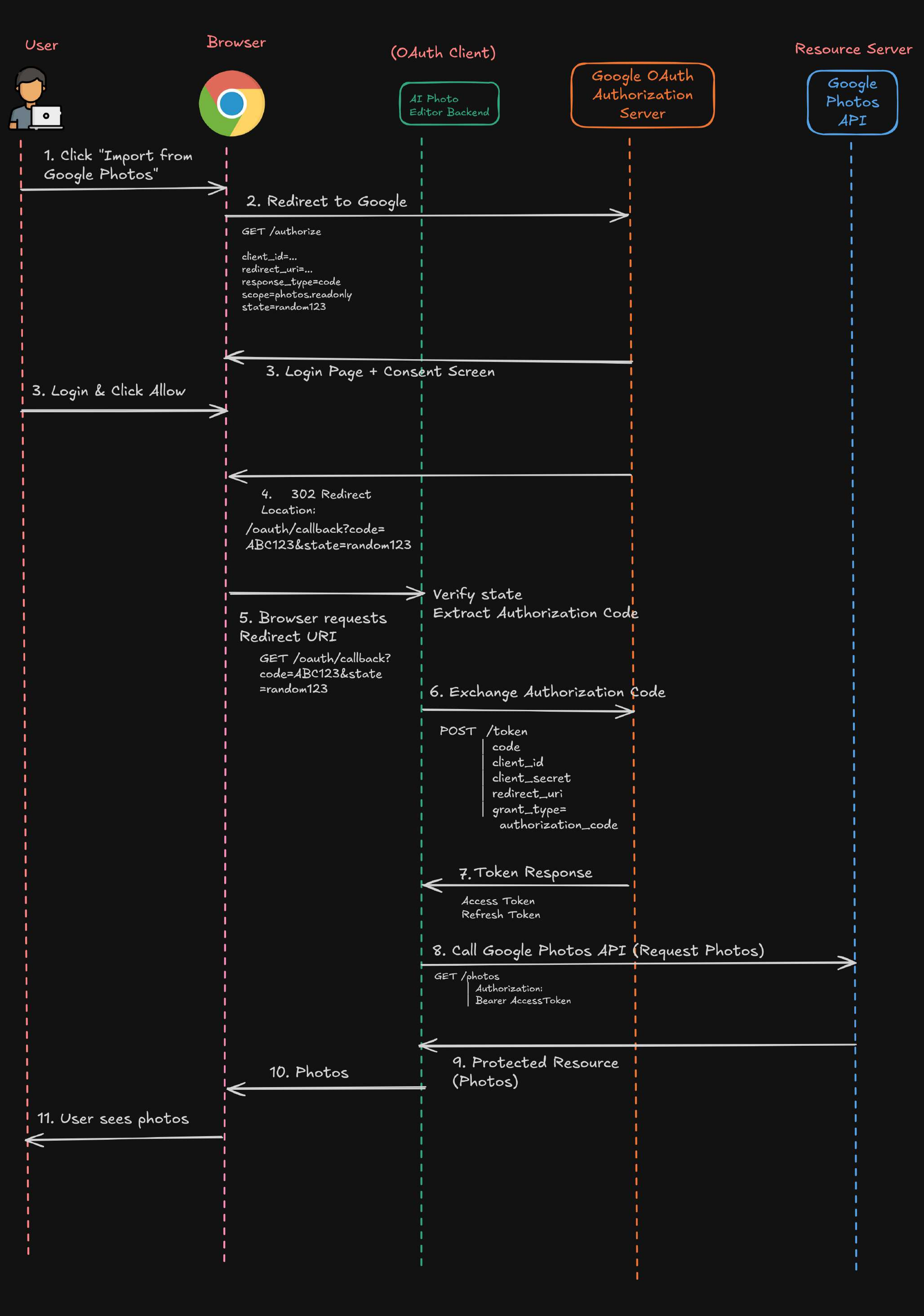
Access Token
Refresh Token

Access Token

Properties:

temporary
limited
scoped
revocable

OAuth 2.0 Authorization Code Flow


User
Browser
(OAuth Client)
AI Photo Editor Backend
Google OAuth Authorization Server
Resource Server
Google Photos API
1. Click "Import from Google Photos"
2. Redirect to Google
GET /authorize
client_id=...
redirect_uri=...
response_type=code
scope=photos.readonly
state=random123
3. Login Page + Consent Screen
3. Login & Click Allow
4. 302 Redirect
Location:
/oauth/callback?code=ABC123&state=random123
Verify state
Extract Authorization Code
5. Browser requests Redirect URI
GET /oauth/callback?code=ABC123&state=random123
6. Exchange Authorization Code
POST /token
code
client_id
client_secret
redirect_uri
grant_type=authorization_code
7. Token Response
Access Token
Refresh Token
8. Call Google Photos API (Request Photos)
GET /photos
Authorization:
Bearer AccessToken
9. Protected Resource (Photos)
10. Photos
11. User sees photos

Browser
-> Redirects User
-> Receives Authorization Code

Backend
-> Exchanges Code
-> Stores Tokens
-> Calls Google APIs

Google
-> Authenticates User
-> Issues Tokens
